CREDIT CARD SERVICE PORTAL (CCSP)

Comprehensive DevOps Lab & Selenium Test Automation - Viva Preparation Guide

Student Name: Mubina	**Roll Number:** 23102A0039	**Program:** B.E. Computer Engineering (Sem VII)
Subject: DevOps Lab	**Evaluator:** Dr. Snehal Andhare	**Assigned Project:** Selenium Testing Framework for CCSP

1. Executive Summary & Problem Definition (Task 1 Context)

The **Credit Card Service Portal (CCSP)** is an enterprise-grade retail banking web application engineered to solve critical operational bottlenecks in traditional financial onboarding. In conventional setups, credit card evaluation relies on manual underwriting, siloed verification documents, and disjointed lifecycle management, leading to high processing latency (3-5 business days), untracked status transitions, and severe deployment friction. CCSP introduces an end-to-end automated digital lifecycle powered by a robust **DevOps CI/CD pipeline** and **Selenium automated quality gates**.

Core Objective: Guarantee zero-defect releases through continuous integration, automated UI regression testing, multi-stage Docker containerization, and idempotent Ansible configuration management with zero downtime.

2. Full Technology Stack & Engineering Justification

Tier / Function	Technology Chosen	Version	Engineering Rationale for Viva Defense
Backend Framework	Java LTS + Spring Boot	Java 17 / 3.2.5	High performance, mature dependency injection, native Actuator observability.
Database Engine	Embedded H2 Database	In-Memory	Zero external database setup, fast in-memory execution, isolated CI/CD test runs.
Frontend Presentation	Thymeleaf + Bootstrap 5	5.3.3	Server-rendered responsive UI; deterministic HTML IDs for Selenium selectors.
Automated Quality Gate	Selenium WebDriver + JUnit 5	4.19 / 5.10	Automates 4 critical user journeys; runs headless in CI with failure screenshot capture.
Build Automation	Apache Maven	3.9.6	Standardized build lifecycle (compile, test, package); automated plugin management.
Containerization	Docker Multi-Stage Build	Engine 24+	Build stage compiles JAR; minimal Eclipse Temurin JRE runtime image (<150 MB).
Continuous Integration	Jenkins Declarative Pipeline	LTS 2.440+	Pipeline-as-Code (Jenkinsfile), automated quality gates, Docker tag and push.
Configuration Mgmt	Ansible Core Playbooks	2.15+	Idempotent host provisioning (changed=0), health probes, automated rollback.
Observability & Telemetry	Spring Boot Actuator	3.2.5	Real-time JSON health telemetry (/actuator/health) and visual UI (/health).

3. The 5 Core Functional MVP Modules (Scope Freeze)

- **1. Digital Onboarding Portal (/apply):** Interactive application capture (Name, Email, Annual Income, Card Tier, Requested Limit, Employment). Enforces strict bean validation gates (@Min income INR 1,00,000, email regex).
- **2. Searchable Master Directory (/cards):** Live multi-criteria filtering across customer names and card numbers. Dynamic status filtering (PENDING, ACTIVE, BLOCKED, REJECTED) with real-time responsive UI rendering.
- **3. Role-Based Lifecycle State Machine:** Strict legal state transitions: PENDING → ACTIVE (Approval) or REJECTED; ACTIVE → BLOCKED (Security Freeze) or restored. Illegal bypasses are rejected by backend business rules.
- **4. Executive KPI Metrics Dashboard (/):** Real-time executive metrics summary cards calculating Total Accounts, Active Cards, Underwriting Queue, and Security Frozen accounts, alongside recent activity audit logs.
- **5. Production Readiness Telemetry (/health & /actuator/health):** Microservice health probes exposing real-time database connectivity, JVM memory statistics, and process uptime for automated CI/CD gating.

4. System Architecture & 4-Tier Monolithic Design (Task 3 Context)

CCSP is engineered with a strict **4-Tier Monolithic Modular Architecture** adhering to the Single Responsibility Principle and Separation of Concerns. This architecture ensures high cohesion, loose coupling, and testability:

Architectural Tier	Key Java / System Components	Responsibilities & Flow
Tier 1: Presentation Layer	Thymeleaf HTML5, Bootstrap 5, CSS3, DOM IDs	Server-renders views (dashboard, cards, apply, health). Exposes deterministic IDs for Selenium locators.
Tier 2: Controller / Web API	CreditCardController.java, CreditCardApiController.java	Handles HTTP verbs (GET, POST, PUT), processes form submissions, performs model binding and content negotiation.
Tier 3: Service / Business Logic	CreditCardService.java, CardType.java, CardStatus.java	Enforces underwriting eligibility gates, 40% income cap, masked card generation, and state transitions.
Tier 4: Persistence / Data Access	CreditCardRepository.java, Spring Data JPA, H2	JPA Object-Relational Mapping, H2 in-memory transactions, seed data insertion on startup.

5. Core Business Rules & Underwriting Guardrails (Viva Highlight)

- **40% Income Multiplier Rule:** A customer's maximum allowable credit limit is strictly capped at **40% of their verifiable annual income** (e.g., Annual Income INR 10,00,000 → Maximum eligible limit = INR 4,00,000).
- **Card Tier Ceiling Guardrails: Platinum:** Max INR 5,00,000 (Premium tier) | **Gold:** Max INR 2,50,000 (Mid tier) | **Titanium:** Max INR 1,00,000 (Entry tier). If calculated limit exceeds tier ceiling, it is capped at the tier maximum.
- **Minimum Income Gate:** Income strictly validated via @Min(100000). Submissions below INR 1,00,000 trigger inline validation errors.
- **Card Number Masking & Audit Trail:** Automatically generates 16-digit card numbers formatted as ****-****-****-XXXX preserving PCI-DSS privacy standards alongside ISO 8601 creation timestamps.
- **Finite State Lifecycle Engine:** Transitions strictly enforced: PENDING → ACTIVE, PENDING → REJECTED, ACTIVE → BLOCKED, BLOCKED → ACTIVE. Prevents invalid illegal status corruptions.

6. End-to-End DevOps CI/CD Lifecycle & Automation Pipeline

The project implements a complete, closed-loop DevOps delivery lifecycle connecting developer commits to verified production deployments:

DevOps Stage	Tool / Technology	Key Actions & Automated Validations Performed
1. Code & Branch	Git / GitHub	Feature branches (feature/*), Conventional Commits, Pull Requests to dev, conflict resolution.
2. Continuous Integration	Jenkins Declarative	SCM checkout, Maven build (mvn clean compile), JAR archive packaging, surefire reports.
3. Continuous Testing	Selenium + JUnit 5	Headless Chrome execution of 4 user journeys; Quality Gate stops pipeline on failure.
4. Containerization	Docker Multi-Stage	Multi-stage Dockerfile: Maven builds JAR, Eclipse Temurin JRE runs it. Image tagged with build number.
5. Continuous Delivery	Jenkins + Docker Registry	Automated container replacement (ccsp-live), port binding 8080:8080, zero-downtime swap.
6. Configuration Mgmt	Ansible Core	Playbook installs prerequisites, verifies ports, enforces idempotency (changed=0 on 2nd run).
7. Reliability & Rollback	Actuator + Ansible	Probes /actuator/health for UP status. Automatic rollback restores prior stable container tag on error.

7. How to Run the Project (Step-by-Step Viva Execution Guide)

Follow these exact commands to execute the project during the practical viva demonstration:

Option A: Run Locally via Maven Wrapper / Installed JDK

```
$env:JAVA_HOME = "C:\Users\YOCKET-271\tools\jdk-17.0.12+7"
$env:PATH = "$env:JAVA_HOME\bin;C:\Users\YOCKET-271\tools\apache-maven-3.9.6\bin;$env:PATH"
cd app
mvn spring-boot:run
```

Option B: Run Automated Unit & Selenium Quality Gate Tests (14 Tests)

```
cd app
mvn test   # Executes 10 unit tests + 4 Selenium browser journeys headless in ~20s
```

Option C: Build & Run via Docker Multi-Stage Container

```
docker build -t mubina23102a0039/credit-card-app:1.0.0 .
docker run -d --name ccsp-live -p 8080:8080 mubina23102a0039/credit-card-app:1.0.0
docker ps && docker logs --tail 20 ccsp-live
```

Option D: Execute Idempotent Ansible Provisioning & Rollback

```
ansible-playbook -i ansible/hosts.ini ansible/playbook.yml         # Full deployment
ansible-playbook -i ansible/hosts.ini ansible/playbook.yml         # 2nd run: changed=0 (Idempotency)
ansible-playbook -i ansible/hosts.ini ansible/playbook_task14.yml --tags rollback # Disaster Recovery
```

8. Verified Project Endpoints Matrix

Endpoint URL	HTTP Verb	Response Content	Verification Purpose
http://localhost:8080/	GET	HTML5 (Thymeleaf)	Executive KPI Dashboard & summary counters.
http://localhost:8080/cards	GET	HTML5 (Thymeleaf)	Master Directory, real-time search & status dropdown filter.
http://localhost:8080/apply	GET / POST	HTML5 (Form)	Digital Onboarding form with bean validation feedback.
http://localhost:8080/health	GET	HTML / JSON	Visual Health Dashboard & REST endpoint for browser/curl.
http://localhost:8080/actuator/health	GET	JSON (Actuator)	Telemetry probe verifying DB, disk, and status: UP.
http://localhost:8080/api/cards	GET / POST	REST JSON	Programmatic REST API for credit card records.
http://localhost:8080/h2-console	GET	HTML Console	H2 In-Memory DB management (jdbc:h2:mem:ccsp_db).

9. Selenium Automated Quality Gate Deep Dive (Tasks 9 & 10)

Selenium WebDriver 4.x is integrated with JUnit 5 to serve as an **uncompromising Quality Gate** in the Jenkins pipeline. Configured via WebDriverManager.chromedriver().setup() and runs headless in CI environments.

The 4 Critical Automated User Journeys:

- **Journey 1 (Digital Application & Directory Sync):** Navigates to /apply, fills valid customer data ('Aarav Sharma', INR 12L income, Platinum tier), submits form, asserts redirect to /cards, asserts success alert message, and verifies card record appears in the master table with initial status 'Pending'.
- **Journey 2 (Validation Gate Enactment):** Submits form with sub-minimum income (< INR 1,00,000) and asserts submission is halted on /apply, inline error message appears (err-annualIncome), and no database record is created.
- **Journey 3 (Dynamic Search & Filter):** Types 'Priya' into search input, submits search, and asserts all returned rows match query without table corruption.
- **Journey 4 (Role-Based Lifecycle Action):** Locates Pending application, triggers 'Approve' action, and asserts status badge transitions to 'Active' with green badge.
- **Failure Screenshot Hook:** Implemented via @AfterEach and TestWatcher. Any assertion failure instantly captures a timestamped PNG into screenshots/failures/.
- **Quality Gate Enforcement:** If any Selenium test fails, Jenkins **halts immediately** and aborts Docker build and deployment.

10. Detailed Breakdown of All 15 DevOps Lab Tasks (Tasks 1 to 8)

Structured summary of problem definitions, technical implementations, and verified deliverables:

Task 1: Problem Definition & Scope

- **Objective:** Study real-time credit card onboarding challenges; identify target users, stakeholders, constraints, and freeze 15-task scope.
- **Implementation:** Drafted formal Problem Statement, stakeholder register (Applicants, Risk Officers, DevOps Engineers), constraints matrix, and 5-module MVP scope freeze.
- **Key Deliverable:** Approved Problem Definition, Stakeholder Matrix, and 5-Module MVP Scope document.

Task 2: Agile Planning & DevOps Workflow

- **Objective:** Create user stories with INVEST criteria, acceptance scenarios (Gherkin), product backlog, Kanban board, DoD, and DevOps lifecycle diagram.
- **Implementation:** Engineered 6 epics, 15 user stories with Given-When-Then criteria, 3-sprint release schedule, strict Definition of Done, and full DevOps lifecycle schematic.
- **Key Deliverable:** Product Backlog, Sprint Plan, 4-tier DoD, and DevOps Workflow architecture diagram.

Task 3: Requirements & Architecture Setup

- **Objective:** Design minimum 4-tier monolithic architecture for CCSP; select tech stack, build tool, database, and local setup.
- **Implementation:** Designed modular architecture with Spring Boot 3.2.5, H2 in-memory DB, Thymeleaf, and Maven; implemented Actuator telemetry and seed data generation.
- **Key Deliverable:** SRS specification, Use-Case diagram, 4-Tier Architecture schematic, and working local build.

Task 4: Git & GitHub Repository Init

- **Objective:** Initialize Git repository; configure README, .gitignore, branch naming rules, issue templates, and commit skeleton.
- **Implementation:** Created GitHub repo with Git Flow branching policy (main, dev, feature/*), Conventional Commits, bug/feature issue templates, and initial Maven skeleton.
- **Key Deliverable:** Repository URL, .gitignore, issue templates, branching rules, and initial commit log.

Task 5: Feature Development & Branching

- **Objective:** Implement Feature 1 (Application Onboarding & Validation) on a feature branch; demonstrate commit, push, PR, review, and merge.
- **Implementation:** Created branch feature/apply-validation; coded DTO, validation gates, controller, and Thymeleaf form; opened PR #1, conducted code review, and merged to dev.
- **Key Deliverable:** PR #1 evidence, code review comments, Git reflog, and verified commit graph.

Task 6: MVP Completion & Git Collaboration

- **Objective:** Implement remaining MVP functions; demonstrate a second feature branch, intentional merge conflict resolution, tagging, and v1.0.0 release.
- **Implementation:** Built cards directory, search, state machine, and KPI dashboard; created intentional merge conflict on README/Service, manually resolved conflict markers, and tagged v1.0.0.
- **Key Deliverable:** Resolved conflict diff (<<<<<<<, =======, >>>>>>>), tag v1.0.0 release, and Git log.

Task 7: Jenkins Installation & CI Job

- **Objective:** Install Jenkins; configure JDK 17 and Maven 3.9; create Freestyle CI build job triggered by commit or polling; archive artifact.
- **Implementation:** Installed Jenkins LTS (WAR); configured tools; created Freestyle job 'credit-card-portal-ci'; configured SCM polling; executed mvn clean package; archived JAR.
- **Key Deliverable:** Jenkins build console log (#1 SUCCESS), archived JAR artifact (credit-card-app-1.0.0.jar).

Task 8: Pipeline as Code (Jenkinsfile)

- **Objective:** Create declarative Jenkinsfile with checkout, compile, quality gate, package, and deployment stages; parameterize environment settings.
- **Implementation:** Authored root Jenkinsfile with 6 declarative stages, parameterized variables (DEPLOY_ENV, SERVER_PORT), automated Tomcat staging run, and /actuator/health probe.
- **Key Deliverable:** Declarative Jenkinsfile, successful pipeline run log, and live health verification.

11. Detailed Breakdown of All 15 DevOps Lab Tasks (Tasks 9 to 15)

Structured summary of testing, containerization, CD pipeline, Ansible provisioning, and final release:

Task 9: Selenium Test Design & Execution

- **Objective:** Design Selenium test suite for 4 critical user journeys with assertions, test data, and automatic failure screenshot capture.
- **Implementation:** Coded CreditCardPortalSeleniumTest.java using Selenium 4.x + JUnit 5 + WebDriverManager; implemented 4 journeys; added @AfterEach screenshot hook for failures.
- **Key Deliverable:** Automated test suite (14 tests passed), local test execution report, and screenshot hook.

Task 10: Continuous Testing in Jenkins Quality Gate

- **Objective:** Integrate Selenium suite into Jenkins pipeline; configure test reporting; demonstrate intentional defect failure and subsequent fix.
- **Implementation:** Integrated JUnit/Selenium execution into Jenkins; introduced intentional bug (income gate inversion) → pipeline failed and halted Docker build; fixed bug → pipeline passed.
- **Key Deliverable:** Surefire XML test report, failed pipeline log (Build #2), bug-fix commit, and successful rerun (#3).

Task 11: Dockerfile & Container Lifecycle

- **Objective:** Create multi-stage Dockerfile; build, tag, and run container; document complete container lifecycle (run, stop, inspect, restart, rm).
- **Implementation:** Authored multi-stage Dockerfile (Maven builder + Eclipse Temurin JRE minimal runtime); verified port mapping 8080:8080; executed and documented complete lifecycle.
- **Key Deliverable:** Dockerfile, lightweight image (<150 MB), Docker CLI command log, and container inspection.

Task 12: Jenkins-Docker Continuous Deployment

- **Objective:** Extend Jenkins pipeline to build versioned Docker image, push to registry, and deploy fresh container automatically post-testing.
- **Implementation:** Added Docker build, tag (IMAGE_TAG=1.0.0.${BUILD_NUMBER}), publish, and deployment stages to Jenkinsfile; implemented zero-downtime container swap and health probe.
- **Key Deliverable:** End-to-end commit-to-container pipeline, versioned Docker image, and live container deployment.

Task 13: Configuration Management with Ansible

- **Objective:** Identify server prerequisites (packages, user, directories, ports); author Ansible inventory and idempotent YAML playbook.
- **Implementation:** Engineered ansible/hosts.ini and ansible/playbook.yml; managed Java 17, Docker, /opt/credit-card-portal directory, user permissions, and port 8080 binding.
- **Key Deliverable:** Ansible inventory, structured YAML playbook, syntax check log, and execution transcript.

Task 14: Automated Provisioning & Disaster Rollback

- **Objective:** Execute Ansible provisioning; verify idempotency (changed=0 on 2nd run); perform health checks; demonstrate automated rollback to prior tag.
- **Implementation:** Executed playbook_task14.yml; achieved changed=0 on rerun (100% idempotent); verified 4 endpoints; executed Play 2 rollback restoration to image tag 1.0.0.4 in 18s.
- **Key Deliverable:** Idempotency log (changed=0, failed=0), multi-endpoint health report, and automated rollback evidence.

Task 15: Final Release Pipeline & Viva Readiness

- **Objective:** Execute full end-to-end DevOps workflow; compile complete technical documentation, troubleshooting guide, and prepare viva defence.
- **Implementation:** Executed seamless commit-to-production pipeline across Git, Jenkins, Selenium, Docker, and Ansible; authored comprehensive lab reports and verified zero regressions.
- **Key Deliverable:** Final production-ready repository, 15 complete lab task reports, whitepaper, and live demo.

12. Top 20 High-Yield Viva Questions & Examiner Defense Answers

Review these precise answers to master the conceptual and practical defense during your viva examination:

Q1: Why did you choose Spring Boot with embedded H2 instead of an external MySQL or PostgreSQL database?

Answer: H2 provides an embedded, zero-configuration in-memory database perfectly tailored for automated CI/CD pipelines. It ensures that Jenkins and Selenium test runs remain 100% deterministic, hermetic, and fast without external networking dependencies or state contamination across test suites.

Q2: How does the underwriting risk validation gate work in Task 5?

Answer: The business logic enforces a dual guardrail: (1) Bean Validation (@Min) rejects annual incomes below INR 1,00,000; and (2) CreditCardService calculates the maximum allowable credit limit as exactly 40% of annual income, capped by the card tier maximum (Platinum INR 5L, Gold INR 2.5L, Titanium INR 1L).

Q3: What is the difference between Git Merge and Git Rebase? How did you resolve the conflict in Task 6?

Answer: Git merge preserves historical context with a non-linear merge commit, whereas rebase rewrites commit history linearly. In Task 6, an intentional conflict was injected on a shared file; Git placed conflict markers (<<<<<<< HEAD ... ======= ... >>>>>>>). I manually resolved the text, ran git add, and committed the clean resolution.

Q4: Why is Selenium headless mode crucial for CI/CD pipelines in Jenkins?

Answer: CI build agents and Docker runners operate in headless Linux/Windows server environments without an attached graphical display (X-server / Windows Desktop GUI). Headless mode (--headless=new) enables full Chrome DOM rendering and JavaScript execution purely in system memory.

Q5: How does Jenkins enforce the Quality Gate in Task 10? What happens when a test fails?

Answer: In Jenkinsfile, stage('Automated Quality Gate') executes 'mvn test'. If any JUnit assertion or Selenium test fails, the Maven process returns a non-zero exit code. Jenkins intercepts this failure, immediately terminates pipeline execution, and aborts Docker build and deployment.

Q6: Explain the multi-stage Docker build architecture in Task 11. Why is it superior to single-stage?

Answer: Stage 1 (Maven Builder) uses a full JDK and Maven image to compile and package the application JAR. Stage 2 (Runtime) copies only the compiled JAR into a minimal Eclipse Temurin JRE runtime image (<150 MB). This eliminates build tools and source code from production, minimizing image size and attack surface.

Q7: What is Ansible Idempotency and how did you verify it in Task 14?

Answer: Idempotency guarantees that applying an automation playbook multiple times yields the exact same target system state without unintended changes or side effects. On the first execution, Ansible installed packages and started containers (changed=5); on the second execution, it verified all states were satisfied and reported 'changed=0, failed=0'.

Q8: How does the automated rollback mechanism in Task 14 ensure disaster recovery?

Answer: If the newly deployed container fails the post-deployment health probe (/actuator/health), Ansible executes Play 2 (tagged 'rollback'). It gracefully stops and removes the unhealthy container and instantly restarts the previous verified stable image tag (1.0.0.4) within 18 seconds.

Q9: What is the purpose of Spring Boot Actuator in enterprise DevOps?

Answer: Spring Boot Actuator provides production-ready operational telemetry over HTTP. The /actuator/health endpoint aggregates component health (disk, database, liveness) into structured JSON used by Docker HEALTHCHECK directives, Jenkins probes, and Kubernetes readiness gates.

Q10: What is the difference between a Freestyle Job (Task 7) and a Declarative Pipeline (Task 8)?

Answer: A Freestyle job is configured via the Jenkins Web GUI and stored in Jenkins internal XML, making version control and disaster recovery difficult. A Declarative Pipeline stores the build workflow as code (Jenkinsfile) in the Git repository, enabling versioning, code reviews, and reproducible stages.

Q11: How does WebDriverManager benefit automated test maintenance?

Answer: Traditional Selenium requires manually downloading and managing browser drivers matching the host's installed browser version. WebDriverManager automatically detects the installed browser (Chrome/Edge), downloads the compatible driver binary, and caches it dynamically.

Q12: How does the role-based status lifecycle prevent invalid state transitions?

Answer: The CreditCard entity enforces a strict finite state machine via switch cases in CreditCardService. Applications can only transition from PENDING → ACTIVE or REJECTED. Only ACTIVE cards can be FROZEN (BLOCKED). Any invalid transition attempt throws an IllegalStateException.

Q13: What happens if an applicant requests a credit limit higher than their eligible ceiling?

Answer: The application does not fail completely; instead, the system caps the granted credit limit at the customer's maximum eligible ceiling (40% of income or tier maximum), logs a warning flash message, and creates the card record in PENDING status for underwriter review.

Q14: Explain the difference between Docker run, stop, restart, and rm.

Answer: docker run creates and starts a new container instance from an image; docker stop sends a SIGTERM followed by SIGKILL to gracefully shut down the container; docker restart halts and re-boots the container; and docker rm permanently removes the stopped container and its writable layer.

13. Essential Command Cheatsheet for Viva Demonstration

Quick reference table of every terminal command used across all 15 tasks:

Category	Command Line Snippet	Description / Expected Output

Git Flow	`git checkout -b feature/apply-validation`
	`git commit -m "feat: add onboarding form"`
	`git merge --no-ff feature/apply-validation`
	`git tag -a v1.0.0 -m "Release v1.0.0"`
		Branch creation, conventional commits, pull request merge, and release version tagging.

Maven Build	`mvn clean compile`
	`mvn test`
	`mvn package -DskipTests=true`
	`mvn spring-boot:run`
		Compiles Java 17 source, executes 14 unit/Selenium tests, builds executable JAR, launches server on 8080.

Selenium Tests	`mvn test -Dtest=CreditCardPortalSeleniumTest`	Executes 4 headless browser journeys in Chrome; saves failure PNGs to screenshots/failures/.

Docker Ops	`docker build -t credit-card-app:1.0.0 .`
	`docker run -d -p 8080:8080 --name ccsp credit-card-app:1.0.0`
	`docker ps && docker logs -f ccsp`
	`docker stop ccsp && docker rm ccsp`
		Multi-stage build, container run with port binding 8080:8080, log inspection, and container deletion.

Ansible Ops	`ansible-playbook -i ansible/hosts.ini ansible/playbook.yml` `ansible-playbook -i ansible/hosts.ini ansible/playbook_task14.yml --tags rollback`	Provisions node prerequisites, verifies idempotency (changed=0), tests automated rollback to tag 1.0.0.4.
Telemetry Probe	`curl -s http://localhost:8080/actuator/health` `curl -s http://localhost:8080/health`	Returns JSON status: UP, confirming embedded database connectivity and runtime health.

14. Evaluator Rubric & Academic Alignment (Dr. Snehal Andhare)

Every requirement outlined in the University Syllabus and Department Assessment Rubric has been fulfilled with zero compromises:

- **Academic Integrity:** Complete consistency across student identity (Mubina, 23102A0039, B.E. Computer Engineering Sem VII).
- **Breadth of DevOps Tools:** Demonstrates complete proficiency in Git, GitHub, Jenkins, Selenium, Docker, and Ansible.
- **Zero-Defect Quality Assurance:** 14 automated JUnit and Selenium tests pass with 100% success rate.
- **Production Reliability:** Enforces idempotent configuration management (changed=0) and automated disaster rollback in 18s.

15. Final Student Declaration & Project Sign-Off

"I hereby declare that this project titled 'Selenium Testing Framework for a Credit Card Service Portal' represents my original laboratory work completed under the academic guidance of Dr. Snehal Andhare. All source code, Docker images, Jenkins pipelines, Ansible playbooks, and automated Selenium test scripts have been independently developed, executed, and verified."

Student Signature: _____
Mubina (Roll No: 23102A0039)
Date: 22nd October 2026

Evaluator Signature: _____
Dr. Snehal Andhare (Subject In-charge)
Department of Computer Engineering